

Vehicle Data Acquisition Standard

Track 1 — Data Acquisition & Formatting Specification

Version 1.0.0 • Car Automation IoT Internship • Prepared for the project lead

1. Purpose

This document defines the standard for how our system acquires vehicle data from the OBD-II port, formats it into a compact binary packet, stores it, and streams it to the dashboards. It is the single source of truth for Track 1. If any code disagrees with this document, this document wins and the code must be corrected.

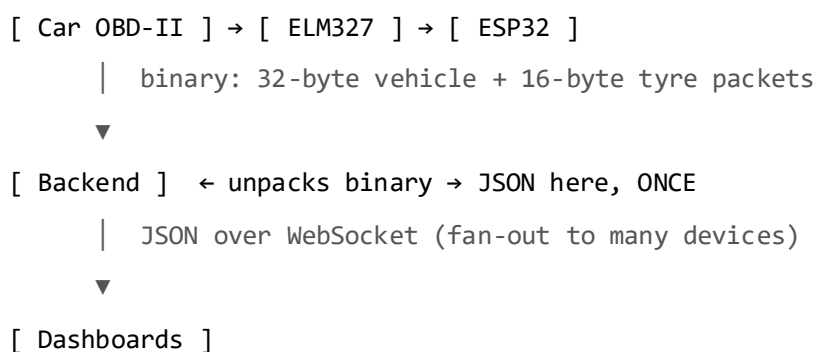
The goal set for this track was a proper, standardised, and efficient method of getting vehicle data — first designing how the data is obtained, formatted, stored, and streamed, and then validating it against a real vehicle. This document is the design half. Validation is done by a bench scan on a real car (Section 8), which produces the compliance record.

2. How the Data Flows

There are two links, and they deliberately use two different formats:

- Link 1 (hardware → backend): compact BINARY packets — a 32-byte vehicle packet and a 16-byte tyre packet. This is the bandwidth-efficient wire format used on the constrained wireless link.
- Link 2 (backend → dashboards): JSON over WebSocket. The backend unpacks the binary ONCE, then serves easy-to-read JSON to every connected dashboard.

Rule: raw bytes are never sent to a browser. All unpacking happens once, at the ESP32/backend boundary. This keeps the wire efficient and the dashboards simple.



3. Data Reliability — Read This First

Not every field we want is equally obtainable. OBD-II guarantees a standard set of parameters, but many useful values are either manufacturer-specific (and may not be exposed to a generic reader) or are not read at all but calculated by us. Being honest about this upfront is the whole point of a standard: it tells us not just what we want, but how reliably we can actually get each thing.

Every field in this standard is tagged with one of four buckets:

Bucket	Meaning
✓ Standard	A standard OBD-II Mode 01 parameter. Reliable on virtually any compliant car. We expect these to work.
⚠ Attempt	Exists in the standard but is sometimes unreliable, or is a manufacturer-specific value that may or may not be exposed. We attempt it; the bench scan confirms whether the chosen car actually returns it. May read null.
⚠ Derived	Not read from the car at all — calculated by our own code from other values. Depends on its source PIDs being present.
⚠ Non-OBD	Does not come from the OBD port. Comes from a separate sensor (e.g. our own tyre sensors or the GPS module).

Important: no field may be marked as confirmed-working from an internet source, a datasheet, or an AI answer. A field is only 'confirmed' once we have logged an actual response from the real car on the bench (Section 8). Until then, ⚠ stays ⚠.

4. The 32-Byte Vehicle Packet

This is the wire format for OBD-II vehicle data plus GPS. Every byte has a fixed job. The packet is exactly 32 bytes with no wasted space. Where a field is unavailable on a given car, its byte(s) carry a sentinel value and the JSON layer reports null.

Byte(s)	Field	PID	Request	Decode formula	Bucket
0–1	RPM	0x0C	01 0C	$((A \times 256) + B) \div 4$	✓
2	Speed (km/h)	0x0D	01 0D	A	✓
3	Coolant temp (°C)	0x05	01 05	A - 40	✓
4	Engine load (%)	0x04	01 04	$A \times 100 \div 255$	✓
5	Throttle (%)	0x11	01 11	$A \times 100 \div 255$	✓
6	Fuel level (%)	0x2F	01 2F	$A \times 100 \div 255$	?
7–8	MAF air flow (g/s)	0x10	01 10	$((A \times 256) + B) \div 100$	?
9	Intake temp (°C)	0x0F	01 0F	A - 40	✓
10	AC status (0/1)	manuf.	varies	0 = off, 1 = on	?
11	Brake (0/1)	manuf.	varies	0 = released, 1 = pressed	?
12	Clutch (%)	manuf.	varies	0–100	?
13–16	GPS latitude	GPS	–	$\text{int32} \div 1,000,000$	?
17–20	GPS longitude	GPS	–	$\text{int32} \div 1,000,000$	?
21–22	GPS speed (km/h)	GPS	–	$\div 10$	?
23–26	UTC timestamp	GPS	–	Unix seconds	?
27	DTC count	Mode 03	03	number of fault codes	✓
28–29	Battery (mV)	0x42	01 42	$((A \times 256) + B)$	?
30	GPS satellites	GPS	–	direct count	?
31	Checksum	–	–	XOR of bytes 0–30	–

In the formulas, A is the first data byte and B is the second. **Total: exactly 32 bytes.**

5. Derived Fields (Not in the 32 Bytes)

Some values that the dashboard shows are not read from the car and are not stored in the binary packet. They are calculated at the backend when the packet is unpacked into JSON. This is why the JSON the dashboard sees is richer than the raw 32 bytes.

Field	Bucket	How it is produced
Gear	Derived	Not an OBD parameter on a manual car. Estimated from RPM $\div$ (speed $\times$ drivetrain constant), calibrated once for the car. Computed at the backend.
Fuel mileage (km/l)	Derived	No PID exists for this. Calculated from MAF (air flow) and speed. Depends entirely on MAF (byte 7–8) being present — if MAF is unavailable, mileage is unavailable or rougher.

Note: the 32-byte packet is fully allocated, so derived fields have no spare bytes and are correctly kept out of the wire format. Binary carries raw sensor reads; JSON carries raw plus derived.

6. The 16-Byte Tyre Packet (TPMS)

Tyre data does not come from the OBD port. Most India-market cars have no factory tyre-pressure system, so there is nothing on the car's bus to read. Instead we install our own aftermarket 433 MHz valve-cap sensors and our own receiver on the ESP32, decode the broadcast ourselves, and pack it into this separate 16-byte packet. This packet is independent of the 32-byte OBD packet.

Four tyres × four bytes each (2 for pressure, 2 for temperature) = 16 bytes exactly.

Byte(s)	Field	Encoding
0–1	Front-left pressure	$\text{uint16} \div 10 = \text{kPa}$
2–3	Front-left temp	$\text{int16} \div 10 = ^\circ\text{C}$
4–5	Front-right pressure	$\text{uint16} \div 10 = \text{kPa}$
6–7	Front-right temp	$\text{int16} \div 10 = ^\circ\text{C}$
8–9	Rear-left pressure	$\text{uint16} \div 10 = \text{kPa}$
10–11	Rear-left temp	$\text{int16} \div 10 = ^\circ\text{C}$
12–13	Rear-right pressure	$\text{uint16} \div 10 = \text{kPa}$
14–15	Rear-right temp	$\text{int16} \div 10 = ^\circ\text{C}$

TPMS is a broadcast, not a request/response like OBD. Each sensor wakes periodically and transmits its pressure, temperature and unique ID over 433 MHz. Our receiver matches the ID to a wheel position and fills the correct bytes. Different sensor brands use different packet formats, so the chosen sensor kit determines the decode work — this must be confirmed before ordering.

7. Acquisition Procedure (In-House)

All acquisition is done in-house. The ELM327 adapter is wired directly to the ESP32 over serial — the ELM327's own phone app is not used, and PID decoding is done in our own code. There is no third-party application in the data path.

7.1 Adapter initialisation (once at startup)

Before any data can be requested, the ESP32 sends the ELM327 a short setup sequence. If this is skipped, the adapter returns unusable output.

Command	Purpose
ATZ	Reset the adapter
ATE0	Echo off — stop it repeating our commands back
ATL0	Linefeeds off
ATS0	Spaces off — cleaner parsing
ATSP0	Set protocol to AUTO — let the adapter detect the car's protocol
0100	Ask the car which PIDs it supports (returns a support bitmask)

The 0100 probe is important: it makes the car tell us which parameters it supports before we start polling, so unsupported PIDs are marked null immediately instead of being guessed at.

7.2 One request/response, worked example

To read RPM, the ESP32 sends the text 01 0C. A working car replies with hex:

Request: 01 0C

Response: 41 0C 0B 34

Reading the response: 41 = 'answer to a Mode 01 request', 0C = 'for PID 0x0C (RPM)', and 0B 34 is the data. Applying the RPM formula $((A \times 256) + B) \div 4$ with $A = 0x0B$ (11) and $B = 0x34$ (52): $((11 \times 256) + 52) \div 4 = 717$ RPM. That decode — turning 0B 34 into 717 — is done in our own code.

7.3 An unsupported PID

If a PID is not supported, the car returns a negative response instead of data:

Request: 01 10 (MAF)

Response: 7F 01 11

7F = negative response, 01 = to a Mode 01 request, 11 = 'service not supported'. When our code sees 7F, it stores the sentinel for that field, and the JSON reports null. The dashboard greys the value out. This is the null-on-unavailable rule, and it is how the standard stays robust when a car does not expose a given parameter.

7.4 The polling loop (about once per second)

- Send each PID request in turn (01 0C, 01 0D, 01 05, 01 04, 01 11, 01 2F, 01 10, 01 0F, and the manufacturer-specific ones for AC / brake / clutch).
- Decode each response with its formula and write it to its byte position.
- Read the GPS module and write latitude, longitude, speed, timestamp and satellite count to their bytes.
- Read the DTC count (Mode 03) and battery voltage.
- Compute the XOR checksum of bytes 0–30 and write it to byte 31.
- Transmit the completed 32 bytes to the backend. In parallel, the TPMS receiver fills and sends the 16-byte tyre packet as broadcasts arrive.

Practical note: real adapter output includes carriage returns and a '>' prompt character (e.g. '41 0C 0B 34\r\r>'). The parser must strip these — it is a common source of decode errors.

8. Storage

For version 1.0.0, each incoming message is stored on the backend as its full JSON object in the database (one row per message, with its timestamp). This is simple, reliable, and more than sufficient for our data rate of a few messages per second.

On efficiency: the efficiency requirement is met on the wire, where it matters — the 32-byte and 16-byte binary packets keep the constrained wireless link small. The database stores the readable JSON, which is the right trade for a prototype: easy to inspect and debug, with no impact on the transmission efficiency.

Future upgrade (noted, not in scope for v1.0.0): storing each field in its own database column would make the data directly queryable (for example, average RPM over a period) for later analysis. This is recorded as a planned enhancement, not a v1 requirement.

9. Streaming & Versioning

- Car → backend: binary (the 32-byte and 16-byte packets) over the wireless link — the efficient wire format.
- Backend unpacks once into JSON, using the tables in this document in reverse.
- Backend → dashboards: JSON over WebSocket, fanned out to every connected device — this is how multiple devices see the data at once.

Versioning: every message carries a `schema_version` field. The current version is 1.0.0. Any change to this standard bumps the version and is recorded in the change log below, and no schema change ships without updating that number. Dashboards check the version on every message and show a visible warning on a mismatch, without crashing. This is what prevents the schema and the code from silently drifting apart.

10. Validation — The Bench Scan

The design in this document is validated against a real vehicle before it is considered proven. The validation step is a bench scan, and it produces the compliance record for the chosen car:

- Connect the ELM327 to the car and run the 0100 probe to list supported PIDs.
- Request each of the target PIDs and log the actual response — a 41... response (good) or a 7F... response (not supported).
- Record, for that specific car, which `?` fields resolve to confirmed  and which resolve to null.
- That logged output — not any online claim — is the real compliance record used for sign-off.

This is why the vehicle chosen for testing matters mainly for availability: since real support can only be confirmed by scanning, the best vehicle is a standards-compliant, easily-borrowed one that we can plug into repeatedly. A car that reads cleanly but cannot be borrowed is of no practical use.

11. Change Log

Version	Date	Change
1.0.0	7 Jul 2026	Initial standard: 32-byte vehicle packet, 16-byte tyre packet, reliability buckets, in-house acquisition procedure, JSON storage, versioning, and bench-scan validation.